

Git

Interview Preparation — Deep Dive

Concepts · Commands · Q&A · Real Scenarios

For Java backend developers (1-3 years experience)
Crash-course revision with interview-grade depth

Table of Contents

1. What is Git? Git vs GitHub
2. Git Architecture — 4 Areas
3. First-Time Setup
4. Essential Daily Commands
5. Branching
6. Merge vs Rebase (Critical!)
7. PR Merge Strategies
8. Remote Operations
9. Resolving Merge Conflicts
10. Stashing
11. Undoing Things — Complete Guide
12. .gitignore
13. Tags
14. Cherry-picking
15. Interactive Rebase
16. Branching Strategies
17. Pull Requests
18. Useful Advanced Commands
19. Git Hooks
20. Real Scenarios You'll Be Asked
21. Top Q&A
22. Quick Cheat Sheet

1. What is Git?

Git is a **distributed version control system (DVCS)** created by Linus Torvalds in 2005 (originally to manage the Linux kernel).

Key characteristics

- **Distributed** — every developer has a full copy of the repo with complete history
- **Snapshot-based** — Git stores snapshots of project state, not deltas (like SVN)
- **Branch-friendly** — branches are cheap (just pointers to commits)
- **Integrity** — every change has a SHA-1 hash; history can't be silently changed
- **Fast** — most operations are local

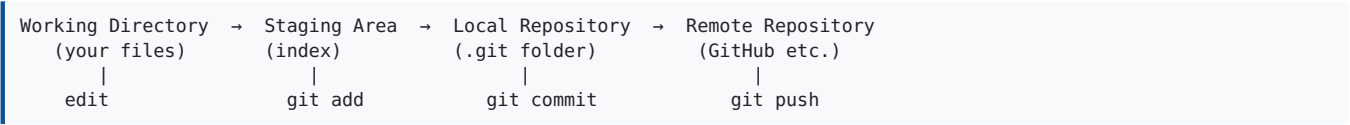
Git vs GitHub

Git	GitHub (or GitLab, Bitbucket)
The version control tool	A hosting service that runs Git remotely
Local	Cloud-hosted
Free, open-source	Platform with PRs, issues, CI/CD, code review

You can use Git without GitHub. GitHub adds collaboration features on top.

2. Git Architecture — 4 Areas

Understanding these is **CRITICAL**. Most Git confusion comes from not knowing where files are.



Area	What's there	How to put files there
Working Directory	Files you edit	(edit them)
Staging Area / Index	Files marked for next commit	<code>git add</code>
Local Repository	Committed history (in <code>.git/</code>)	<code>git commit</code>
Remote Repository	Server (GitHub etc.)	<code>git push</code>

Why a staging area? Lets you review and selectively choose what goes into the next commit. You can edit 5 files but commit only 2 logical changes separately.

3. First-Time Setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global core.editor "vim"          # or "code --wait" for VSCode
git config --global pull.rebase false          # default merge on pull

git config --list                               # view all settings
```

4. Essential Daily Commands

Repository setup

```
git init                # create new repo in current dir
git clone <url>          # download existing repo
git clone <url> myfolder  # clone into specific folder
git remote -v           # list remotes
git remote add origin <url> # add remote
```

Status & inspection

```
git status              # what's changed/staged
git status -s           # short format
git diff                # working dir vs staging
git diff --staged       # staging vs last commit
git diff HEAD           # working dir vs last commit
git diff <branch1> <branch2> # compare branches
git log                 # commit history
git log --oneline       # condensed
git log --graph --all --oneline # visual branches
git log -p              # show actual diffs
git log --author="Alice"
git log --since="2 weeks ago"
git show <commit-hash>  # details of a commit
```

Staging & committing

```
git add file.txt        # stage one file
git add .               # stage everything in current dir
git add -A              # stage all (incl. deleted)
git add -p              # interactive – stage hunks
git restore --staged file.txt # unstage (Git 2.23+)
git reset HEAD file.txt  # older alternative

git commit -m "Add feature"
git commit -am "msg"     # add + commit tracked files
git commit --amend       # edit last commit (message or files)
git commit --amend --no-edit # add staged changes to last commit
```

Undoing changes

```
git restore file.txt    # discard working dir changes
git checkout -- file.txt # older alternative
git restore --staged file.txt # unstage
git reset --soft HEAD~1 # undo last commit, keep changes staged
git reset --mixed HEAD~1 # undo + unstage (default)
git reset --hard HEAD~1 # undo + DISCARD changes (dangerous!)
git revert <commit-hash> # safe undo: creates new commit reversing it
```

5. Branching (the heart of Git)

A **branch** is just a movable pointer to a commit. Creating one is essentially free (no copying of files).

```
git branch                # list local branches
git branch -a             # list all (incl. remote)
git branch -v             # with last commit info
git branch feature        # create branch
git checkout feature      # switch to branch
git switch feature        # newer alternative (Git 2.23+)
git checkout -b feature   # create + switch
git switch -c feature     # newer equivalent
git branch -d feature     # delete (safe – checks if merged)
git branch -D feature     # force delete
git branch -m old-name new-name # rename
```

Typical workflow

```
git switch -c feature/login    # create feature branch from main
# ... edit files ...
git add . && git commit -m "Add login form"
git push -u origin feature/login # push, set upstream tracking
# (open Pull Request in GitHub)
# After merge, locally:
git switch main
git pull
git branch -d feature/login
```

6. Merge vs Rebase (THE classic interview question)

Both integrate changes from one branch into another, but in **different ways**.

Setup — same starting point

```
main:    A → B → C
          \
feature:  D → E
```

git merge — preserves history, creates merge commit

```
git switch main
git merge feature

main:    A → B → C ——— M
          \      /
feature:  D → E ———/
```

- Non-fast-forward merge — creates a merge commit M
- History shows the branching happened
- Easy to spot "this came from a feature branch"
- Can become messy with many merges

git rebase — replays your commits onto another branch

```
git switch feature
git rebase main

main:    A → B → C
          \
feature:  D' → E'   (D and E are reapplied as new commits)
```

- Linear history
- D' and E' have **new commit hashes** (recreated)
- Looks like feature was developed from latest main
- Cleaner history, but **rewrites commit hashes**

Comparison

Aspect	Merge	Rebase
History	Preserves all branches	Linear, "as if" feature came from latest main
Creates merge commit?	Yes	No
Modifies existing commits?	No	Yes (new hashes)
Conflict resolution	Once at end	Per commit (can be many times)
Safe on shared branches?	✓ Yes	✗ No (rewrites history)
Ease of debugging	Easier (real history)	Harder (faked linearity)

The Golden Rule of Rebase: Never rebase commits that have been pushed to a shared branch. If others have pulled them, rewriting will cause chaos.

When to use which?

- **Rebase** for your local feature branch before merging to main → clean history
- **Merge** for shared branches or to preserve historical context

Common workflow

```
# On feature branch
git fetch origin
git rebase origin/main          # bring in latest main, keep linear
# resolve conflicts if any
git push --force-with-lease     # force push (safer than --force)

# Then merge via PR (squash, merge, or rebase merge – depends on team)
```

7. PR Merge Strategies

Modern teams use one of three strategies when merging PRs:

Strategy	Result
Merge commit	Standard merge — keeps full feature history + adds a merge commit
Squash and merge	All feature commits squashed into one commit on main. Clean but loses individual commits.
Rebase and merge	Replays feature commits linearly on main. No merge commit.

Choice depends on team preference. **Squash** is most common for small features (clean main). **Merge commit** preserves detailed history.

8. Remote Operations

```
git fetch                # download remote changes (no merge)
git fetch origin
git pull                # = fetch + merge
git pull --rebase       # = fetch + rebase (cleaner)
git push origin main    # push to remote
git push -u origin feature # set upstream first time
git push --force        # ⚠ dangerous – overwrites remote
git push --force-with-lease # safer – fails if remote moved unexpectedly
```

fetch vs pull

- **fetch** = download new commits but don't apply them
- **pull** = fetch + merge (or rebase) automatically

Use `fetch` to inspect changes before integrating.

9. Resolving Merge Conflicts

When Git can't auto-merge, you'll see:

```
CONFLICT (content): Merge conflict in src/User.java
Automatic merge failed; fix conflicts and then commit the result.
```

File contents look like

```
<<<<<< HEAD
String greeting = "Hello";
=====
String greeting = "Hi there";
>>>>>> feature
```

Resolution steps

1. Open conflicted file(s) — `git status` shows them
2. Edit manually — keep what you want, remove conflict markers
3. `git add <file>` to mark resolved
4. `git commit` (for merge) or `git rebase --continue` (for rebase)
5. Or abort: `git merge --abort` / `git rebase --abort`

Tools

- IDE-based merge tools (IntelliJ, VSCode have great visual conflict resolvers)
- `git mergetool` — launches external tool

10. Stashing (saving work in progress)

When you need to switch branches but have uncommitted changes:

```
git stash                # save uncommitted changes
git stash save "WIP: refactor" # with message
git stash list           # show all stashes
git stash show           # show files in latest stash
git stash apply          # reapply latest stash, keep in stash
git stash pop            # apply + remove from stash
git stash drop           # delete latest stash
git stash clear          # delete all stashes
git stash branch new-branch # create branch from stash
```

When to use

- Pulling latest main while in middle of feature work
- Switching to fix a hotfix on another branch
- Testing alternative approaches without committing

11. Undoing Things — Complete Guide

This is what interviewers love to drill into. Pick the right tool for each situation.

Scenario 1: Discard local changes (not yet staged)

```
git restore file.txt          # Git 2.23+
git checkout -- file.txt      # older
```

Scenario 2: Unstage a file

```
git restore --staged file.txt  # Git 2.23+
git reset HEAD file.txt       # older
```

Scenario 3: Undo last commit, keep changes

```
git reset --soft HEAD~1       # changes staged
git reset --mixed HEAD~1      # changes unstaged (default)
git reset HEAD~1
```

Scenario 4: Undo last commit, throw away changes

```
git reset --hard HEAD~1       # ⚠ permanent
```

Scenario 5: Change last commit message

```
git commit --amend -m "New message"
```

Scenario 6: Add forgotten file to last commit

```
git add forgotten.txt
git commit --amend --no-edit
```

Scenario 7: Undo a published commit (already pushed)

```
git revert <commit-hash>      # creates a NEW commit that undoes it
git push
```

Use `revert`, not `reset`, on pushed commits!

Scenario 8: Recover deleted commits

```
git reflog                    # shows all HEAD movements
git reset --hard HEAD@{2}     # restore to where HEAD was 2 moves ago
```

`reflog` is your safety net — Git keeps deleted refs for 30+ days.

reset vs revert — KEY interview question

Aspect	git reset	git revert
What it does	Moves branch pointer back	Creates new commit that reverses
History	Rewrites (deletes commits)	Preserves (adds undo commit)
Safe on shared branches?	✗ NO	✓ YES
Use case	Undo local mistakes	Undo published mistakes

12. .gitignore

Tells Git which files to ignore (build artifacts, IDE files, secrets).

Sample for Java project

```
# Compiled
target/
build/
*.class

# IDE
.idea/
.vscode/
*.iml

# OS
.DS_Store
Thumbs.db

# Logs & temp
*.log
*.tmp

# Environment
.env
application-local.properties

# Dependencies
node_modules/
```

Common gotcha: Already-tracked files are NOT ignored, even if added later to .gitignore. Remove them first:

```
git rm --cached file.txt
```

13. Tags (versioning)

Tags mark specific commits — typically for releases.

```
git tag                # list tags
git tag v1.0.0         # lightweight tag
git tag -a v1.0.0 -m "Release 1.0" # annotated tag (recommended)
git tag -a v1.0.0 <commit-hash>    # tag specific commit
git push origin v1.0.0             # push specific tag
git push --tags                   # push all tags
git tag -d v1.0.0                 # delete local tag
git push origin :refs/tags/v1.0.0 # delete remote tag
```

Lightweight vs Annotated

- **Lightweight** = just a name pointing at a commit
- **Annotated** = full Git object with author, date, message, optional GPG signature → preferred for releases

14. Cherry-picking

Apply a specific commit from one branch onto another.

```
git cherry-pick <commit-hash>      # apply that commit to current branch
git cherry-pick <hash1> <hash2>    # multiple commits
git cherry-pick --abort             # if conflict
```

Use case

You committed a hotfix on `develop` but it also needs to go to `main` immediately:

```
git switch main
git cherry-pick <hotfix-commit>
```

15. Interactive Rebase (advanced — interview gold)

```
git rebase -i HEAD~5          # last 5 commits
```

Opens an editor:

```
pick a1b2c3 Add login
pick d4e5f6 Fix typo in login
pick g7h8i9 Add tests
pick j1k2l3 Fix typo
pick m4n5o6 Update docs
```

Replace **pick** with

- **reword** — change commit message
- **edit** — pause to amend
- **squash** (s) — combine with previous commit (keeps both messages)
- **fixup** (f) — combine + discard this commit's message
- **drop** — delete the commit
- Reorder lines to reorder commits

Use cases

- Clean up commits before pushing/PR
- Combine "fix typo" commits into the original commit
- Reorder for logical flow

Don't interactive-rebase commits that are pushed to shared branches.

16. Common Branching Strategies

Git Flow (older, comprehensive)

```
master/main → only releases
develop → integration branch
feature/* → branched from develop
release/* → final testing
hotfix/* → emergency fixes from main
```

Heavy. Better for products with scheduled releases.

GitHub Flow (modern, simple)

```
main → always deployable
feature branches → PR → merge → deploy
```

Light. Continuous deployment friendly.

Trunk-Based Development

```
Everyone works on main; short-lived feature branches (< 1 day)
Feature flags for incomplete work
```

Used by Google, Facebook. Requires high test coverage.

GitLab Flow

Hybrid — feature branches + environment branches (`production` , `staging`).

Modern teams mostly use GitHub Flow or trunk-based.

17. Pull Requests / Merge Requests

Not a Git concept — a **GitHub/GitLab/Bitbucket feature** layered on top.

Typical PR workflow

1. Create feature branch
2. Push branch to remote
3. Open PR on GitHub → review, discussion, CI runs
4. Reviewers approve
5. Merge (squash / merge / rebase)
6. Branch deleted
7. CI/CD deploys

Best practices

- Keep PRs small (under 400 lines ideal)
- Descriptive title + description
- Link to ticket/issue
- Self-review before requesting review
- Tests passing
- One logical change per PR
- Respond to feedback promptly

18. Useful Advanced Commands

git blame — who changed this line?

```
git blame file.txt           # show author/commit per line
git blame -L 10,20 file.txt  # specific line range
```

git bisect — binary search through history

```
git bisect start
git bisect bad           # current is broken
git bisect good v1.0     # v1.0 was working
# Git checks out commit halfway between → test it
git bisect good          # or 'bad'
# Repeats until it pinpoints the breaking commit
git bisect reset         # exit
```

git log power flags

```
git log --all --graph --decorate --oneline
git log --grep="bug"           # search messages
git log -S "function_name"     # search added/removed code
git log --author="Alice"
git log --since="2 days ago"
git log path/to/file           # commits touching a file
```

git diff power flags

```
git diff main..feature        # commits in feature not in main
git diff main...feature       # what feature added since branching
git diff --stat               # summary
git diff --word-diff           # word-level diff
```

19. Git Hooks (advanced)

Scripts that run on Git events. Located in `.git/hooks/`.

Hook	Triggers
pre-commit	Before commit (run linters, tests)
commit-msg	Validate commit message format
pre-push	Before push (final tests)
post-merge	After merge (e.g., update dependencies)

Example: pre-commit hook to run tests

```
#!/bin/sh
# .git/hooks/pre-commit
mvn test || exit 1
```

Frameworks: **Husky** (JS), **pre-commit** (Python) standardize this.

20. Real Scenarios You'll Be Asked

"I committed to wrong branch"

```
# On wrong branch
git log                # find commit hash
git reset HEAD~1       # undo, keep changes
git stash              # save them
git switch correct-branch
git stash pop
git commit -am "msg"
```

"I committed sensitive data (passwords)"

1. **NEVER just delete and commit** — it's still in history
2. If unpushed: amend or reset
3. If pushed: rewrite history with `git filter-branch` or **BFG Repo-Cleaner**
4. **Rotate the credential** (assume it's compromised)

"I need to combine my last 3 commits into one"

```
git rebase -i HEAD~3
# Mark first as 'pick', other two as 'fixup'
```

"How to share a single commit with another branch?"

```
git switch other-branch
git cherry-pick <commit-hash>
```

"How to find which commit broke the build?"

```
git bisect start
git bisect bad
git bisect good <last-known-good-commit>
# test, mark good/bad, repeat
```

"How to recover a deleted branch?"

```
git reflog                # find the last commit hash on the branch
git checkout -b recovered-branch <hash>
```

21. Top Q&A

Q: What is Git?

Distributed version control system. Each developer has a complete copy of the repo with full history. Tracks changes via snapshots, supports branching, merging, and remote collaboration.

Q: Git vs GitHub?

Git = the version control tool. GitHub = a hosting service with PRs, issues, CI/CD on top of Git.

Q: What is the staging area / index?

An intermediate area between your working directory and repository. Files added with `git add` are staged for the next commit. Lets you craft commits selectively.

Q: Difference between git pull and git fetch?

`fetch` downloads remote changes without integrating them. `pull` = `fetch` + `merge` (or `rebase`). Use `fetch` to inspect before merging.

Q: Difference between git merge and git rebase?

Merge: combines branches with a merge commit, preserves history. **Rebase:** replays commits on top of another branch, creates linear history but rewrites commit hashes. Rebasing shared branches is dangerous.

Q: When to use rebase vs merge?

Rebase your local feature branch before merging for clean history. Use merge for shared/published branches.

Q: What is the Golden Rule of rebase?

Never rebase commits already pushed to a shared branch. It rewrites history → confuses everyone else.

Q: Difference between git reset and git revert?

reset: moves branch pointer back, rewrites history (dangerous on shared branches). **revert:** creates a new commit that undoes a previous one (safe on shared branches).

Q: Three modes of git reset?

`--soft` (undo commit, keep changes staged), `--mixed` (default — undo + unstage), `--hard` (undo + discard changes — dangerous!).

Q: How do you undo a published commit?

`git revert <hash>` then push. Creates a new commit reversing changes — preserves history, safe for shared branches.

Q: What is git stash?

Temporarily saves uncommitted changes so you can switch branches. Apply later with `git stash pop` or `apply`.

Q: How to recover a deleted commit?

`git reflog` shows all recent HEAD movements. Use `git reset --hard <hash>` to recover. Git keeps refs for ~30 days by default.

Q: What is git cherry-pick?

Apply a specific commit from one branch to another. Useful for porting hotfixes between branches.

Q: What is git bisect?

Binary search through commits to find the one that introduced a bug. Mark a known-good and known-bad commit; Git checks out commits halfway and you mark them as good/bad until pinpointed.

Q: What does HEAD mean?

A pointer to the current commit (typically the tip of your current branch). Detached HEAD = HEAD points directly to a commit, not a branch.

Q: What is a detached HEAD?

HEAD pointing to a commit instead of a branch. Happens when you `git checkout <hash>`. New commits made here are lost when you switch unless you create a branch.

Q: How to resolve a merge conflict?

1) `git status` shows conflicted files. 2) Edit them — remove conflict markers (`<<<`, `===`, `>>>`). 3) `git add` resolved files. 4) `git commit` (or `git rebase --continue`).

Q: What is a fast-forward merge?

When the target branch hasn't diverged — Git just moves the pointer forward. No merge commit created. To force a merge commit: `git merge --no-ff feature`.

Q: Difference between git checkout and git switch?

`switch` (Git 2.23+) is purpose-built for branch switching. `checkout` is overloaded — used for branches, files, commits. Modern Git prefers `switch` and `restore` for clarity.

Q: What is git push --force-with-lease?

Safer than `--force`. Refuses to overwrite the remote if someone else pushed in the meantime — prevents you accidentally clobbering their work.

Q: What happens when you delete a branch with unmerged changes?

`git branch -d` refuses to delete (safe). `git branch -D` force deletes — orphan commits are unreachable but recoverable via `reflog` for ~30 days.

Q: What is .gitignore? Files already committed are still tracked?

Specifies files for Git to ignore. Files already tracked aren't affected — remove with `git rm --cached <file>` first, then commit.

Q: What is origin?

Default name for the remote you cloned from. Just a convention — you can rename or have multiple remotes (`upstream`, etc.).

Q: What's a fork?

A copy of a repo on GitHub under your account. You make changes there, then open a PR to the original. Used for open-source contributions.

Q: What is a tag?

A label pointing to a specific commit. Used for releases (`v1.0.0`). Annotated tags carry metadata (author, message, signature).

Q: How to find who changed a specific line?

`git blame <file>` shows the author, commit, and date of each line.

Q: Squash merge vs merge commit?

Squash: condenses all feature commits into one on main. Clean history but loses individual commits. **Merge commit:** keeps all commits + a merge commit. Detailed history.

Q: What's a Pull Request?

GitHub/GitLab feature for code review. Opens discussion on a branch's changes; reviewers approve, then merge. Not a Git concept.

Q: How to update a feature branch with latest main?

```
git switch feature
git fetch origin
git rebase origin/main      # clean history
# OR
git merge origin/main      # safer for shared branches
```

Q: How to rename a branch?

```
git branch -m old-name new-name      # rename current
git push origin --delete old-name    # delete old on remote
git push -u origin new-name          # push new
```

Q: How to undo git add?

`git restore --staged <file>` (Git 2.23+) or `git reset HEAD <file>`.

Q: Working dir vs Staging vs Repository?

Working dir: files you edit. Staging (index): selected for next commit. Repository (.git): committed history.

Q: What is git log --oneline --graph --all?

Visual log of all branches in compact form. Use it to see your branch topology.

Q: How to set up SSH for Git?

1) `ssh-keygen -t ed25519 -C "you@example.com"` 2) Add public key (`~/.ssh/id_ed25519.pub`) to GitHub settings 3) Use SSH URL: `git@github.com:user/repo.git`

Q: What's the typical Git workflow you use?

"Standard GitHub Flow: I create a feature branch off main, commit logical chunks, rebase on main before opening a PR for clean history, get code review, then squash-merge to keep main history concise. For hotfixes, I branch off main and cherry-pick into other long-lived branches if needed."

22. Quick Cheat Sheet

```
# Daily flow
git status
git add .
git commit -m "msg"
git push

# check state
# stage everything

# New feature
git switch -c feature/xyz
# ... work ...
git commit -am "msg"
git push -u origin feature/xyz

# Update with main
git fetch origin
git rebase origin/main
git push --force-with-lease

# Undo last (local) commit
git reset --soft HEAD~1

# Undo published commit
git revert <hash>

# Discard local changes
git restore <file>

# Recover deleted commits
git reflog
git reset --hard <hash>

# Stash work
git stash
git stash pop

# See history
git log --oneline --graph --all
```

End of Git Interview Prep

Master these commands and Q&A — you'll handle any Git interview question with confidence.